Advanced Geometry

3D hulls, Delaunay/Voronoi, Minkowski sums, and geometry that never breaks

Learning goals

- Choose **exact** predicates (integer / `__int128`) and know their bit budgets
- Implement a robust **incremental 3D convex hull** and argue it is correct
- Get **Delaunay** for free from the 3D hull via the **lifting map**
- Reason about **Voronoi** diagrams as the Delaunay dual (and when to avoid building them)
- Compute **Minkowski sums** of convex polygons in O(n + m) and use them for collision / distance
- Recognise the degenerate cases that kill WF geometry submissions

Where we are

- Week 31 / regional-08: orientation, segments, 2D hull, precision basics
- Nationals-08: rotating calipers, half-plane intersection, closest pair, angular sweep
- **Today:** the next level — dimension 3, duality, and robustness as a design goal
- Next lecture is combinatorics; geometry returns only inside the WF simulation (Lecture 10)

> At WF, geometry problems are often solved by few teams — not because the idea is hard, but because the **implementation must be exactly right**.

Three ways a geometry solution dies

Failure	Typical symptom	Cure
Floating-point error	WA on test 57 of 80	exact integer predicates; eps chosen by analysis
Degeneracy	RE / infinite loop on collinear, coplanar, duplicate input	handle explicitly or perturb
Overflow	WA only on the large tests	count bits *before* coding; `__int128`

Rule: **predicates exact, constructions approximate.** Decide "which side" with integers; compute coordinates of new points with `long double` only at the end.

Bit budgets for exact predicates

Predicate	Degree	coords ≤ 10^6	coords ≤ 10^9
2D orient (cross)	2	~2^{43} ✓ `ll`	~2^{63} — **borderline**
3D orient (triple product)	3	~2^{65} × → `__int128`	~2^{95} `__int128`
2D incircle	4	~2^{86} `__int128`	~2^{126} — **at the limit**
Lifted 3D orient (x, y, x^2+y^2)	4	`__int128`	do not

Degree d with coordinates up to C and differences up to 2C: |value| ≤ (constant) · (2C)^d. Write the bound in a comment.

When you *must* use floating point

- Inputs are reals, or the answer is a constructed point (intersection, circumcenter)
- Use `long double`, compare with **eps scaled to the magnitude** of the operands
- Keep chains short: build constructions from *input* points, not from other constructions
- Symbolic perturbation (Simulation of Simplicity) exists — at WF, prefer explicit case handling
- Snap constructed values back to a grid only if the statement guarantees it

3D toolbox

```
struct P3 { ll x, y, z; P3 operator-(const P3& o) const { return {x-o.x, y-o.y, z-o.z}; } };
// sign of det[b-a, c-a, d-a]; > 0 means d is on the side of normal (b-a)x(c-a)
LL vol(P3 a, P3 b, P3 c, P3 d) {
    P3 u = b - a, v = c - a, w = d - a;
    return (LL)u.x * ((LL)v.y*w.z - (LL)v.z*w.y)
         - (LL)u.y * ((LL)v.x*w.z - (LL)v.z*w.x)
         + (LL)u.z * ((LL)v.x*w.y - (LL)v.y*w.x);
}
```

- Plane through a, b, c: normal n = (b−a)×(c−a), offset n·a
- Signed distance of p to plane: (n·p − n·a)/|n|
- Projection to 2D: drop the coordinate where |n| has its largest component

The 3D convex hull problem

- Input: n points in 3D. Output: triangular faces, outward oriented
- Euler: V − E + F = 2; with triangles 3F = 2E ⇒ **F ≤ 2n − 4, E ≤ 3n − 6**
- Output is linear size, so O(n log n) is optimal; O(n^2) incremental is the contest standard for n ≤ ~3000–5000
- Coplanar faces come out **triangulated** — merge afterwards if the problem counts polygonal faces

Incremental hull: the idea

1. Find 4 non-coplanar points → tetrahedron, faces oriented outward
2. Add points one at a time
3. Face f is **visible** from p if p is strictly on its outer side (`vol > 0`)
4. No visible face ⇒ p is inside (or on) the hull: skip
5. Delete visible faces; the boundary of the visible region is the **horizon**
6. Connect every horizon edge to p

The visible region is always connected (it is the set of faces "seen" from p).

Finding the horizon with edge stamps

```
visible face (a,b,c) owns directed edges a→b, b→c, c→a
its neighbour across a→b owns b→a

horizon edge  ⇔  a→b in a visible face  AND  b→a in a non-visible face
```

- Stamp every directed edge of every visible face with the current step `i`
- Edge a→b is on the horizon iff `stamp[b][a] != i`
- New face (a, b, i) keeps the orientation of the deleted face ⇒ outward automatically
- Memory: n × n ints — fine to n ≈ 4000 (use a hash map beyond)

Code (1/2): initial tetrahedron

```
vector<array<int,3>> hull3d(const vector<P3>& p) {
    int n = p.size(); if (n < 4) return {};
    int i1 = -1, i2 = -1, i3 = -1;
    for (int i = 1; i < n && i1 < 0; i++)            // distinct from p[0]
        if (p[i].x != p[0].x || p[i].y != p[0].y || p[i].z != p[0].z) i1 = i;
    if (i1 < 0) return {};
    for (int i = 1; i < n && i2 < 0; i++) {          // not collinear
        P3 u = p[i1] - p[0], v = p[i] - p[0];
        if ((LL)u.y*v.z - (LL)u.z*v.y || (LL)u.z*v.x - (LL)u.x*v.z ||
            (LL)u.x*v.y - (LL)u.y*v.x) i2 = i;
    }
    if (i2 < 0) return {};
    for (int i = 1; i < n && i3 < 0; i++)            // not coplanar
        if (vol(p[0], p[i1], p[i2], p[i]) != 0) i3 = i;
    if (i3 < 0) return {};
    vector<array<int,3>> F;
    auto addFace = [&](int a, int b, int c, int inside) {
        if (vol(p[a], p[b], p[c], p[inside]) > 0) swap(b, c);
        F.push_back({a, b, c});
    };
    int t[4] = {0, i1, i2, i3};
    for (int k = 0; k < 4; k++) addFace(t[(k+1)%4], t[(k+2)%4], t[(k+3)%4], t[k]);
```

Code (2/2): adding points

```
    vector<vector<int>> stamp(n, vector<int>(n, -1));
    vector<char> used(n, 0);
    for (int k = 0; k < 4; k++) used[t[k]] = 1;
    for (int i = 0; i < n; i++) {
        if (used[i]) continue;
        vector<array<int,3>> keep, vis;
        for (auto& f : F)
            (vol(p[f[0]], p[f[1]], p[f[2]], p[i]) > 0 ? vis : keep).push_back(f);
        if (vis.empty()) continue;
        for (auto& f : vis)
            for (int e = 0; e < 3; e++) stamp[f[e]][f[(e+1)%3]] = i;
        for (auto& f : vis)
            for (int e = 0; e < 3; e++) {
                int a = f[e], b = f[(e+1)%3];
                if (stamp[b][a] != i) keep.push_back({a, b, i});   // horizon
            }
        F.swap(keep);
    }
    return F;
}
```

Why it is correct (and robust)

- **Strict** visibility (`> 0`): points on a face plane but inside the face are skipped; points on the plane but *outside* the face strictly see a neighbouring face (convexity)
- New face (a, b, p) is never degenerate: if p were on line ab, neither face at edge ab would be strictly visible
- Duplicates are inside ⇒ skipped
- All predicates exact ⇒ the combinatorial structure is always a valid closed surface
- Test it: every directed edge has its reverse, all points have `vol ≤ 0` for all faces, V − E + F = 2

Complexity and speed

- Each step scans all faces: $O(n \cdot F)$ = **$O(n^2)$** worst case
- 3000 points on a sphere (worst for F): ~0.1 s in our tests
- Randomised incremental with a conflict graph: expected **$O(n \log n)$** — rarely needed at WF
- Shuffle input first: avoids adversarial insertion orders creating huge intermediate hulls
- Volume = Σ over faces $\text{vol}(O, a, b, c)/6$; area = $\Sigma \, |(b-a)\times(c-a)|/2$

The lifting map

Map $(x, y) \mapsto (x, y, x^2 + y^2)$ onto the paraboloid.

- A circle in the plane lifts to a **plane section** of the paraboloid
- d is inside circle(a, b, c) $\iff$ lifted d is **below** the plane through lifted a, b, c
- incircle(a, b, c, d) = orient3D($\hat{a}$, $\hat{b}$, $\hat{c}$, $\hat{d}$) up to sign

Theorem. The lower faces of the 3D hull of the lifted points project to the **Delaunay triangulation**.

One dimension down: CF 1142C *U2* maps $(x, y) \mapsto (x, y - x^2)$, so parabolas $y = x^2 + bx + c$ become lines and the answer is an upper-hull edge count.

Delaunay from the 3D hull

```
vector<array<int,3>> delaunay(const vector<pair<ll,ll>>& pts) {
    vector<P3> p;
    for (auto [x, y] : pts) p.push_back({x, y, x*x + y*y});
    vector<array<int,3>> res;
    for (auto& f : hull3d(p)) {
        P3 u = p[f[1]] - p[f[0]], v = p[f[2]] - p[f[0]];
        if ((LL)u.x*v.y - (LL)u.y*v.x < 0)          // outward normal points down
            res.push_back({f[0], f[2], f[1]});        // CCW in the plane
    }
    return res;
}
```

- Fails (returns empty) when all points are collinear **or all cocircular** — special-case them
- Coordinates ≤ ~10^6 keep the degree-4 predicate inside `__int128`

What Delaunay gives you

- Empty circumcircle: no input point strictly inside any triangle's circumcircle
- Maximises the minimum angle among all triangulations
- Contains the **Euclidean MST** ⇒ EMST in O(n log n) after triangulation (3n edges, Kruskal)
- Contains every point's **nearest neighbour** edge
- Its dual is the **Voronoi diagram**: Voronoi vertices = circumcentres of Delaunay triangles

Voronoi in contests

- Often you need only a *property* of the Voronoi diagram, not the diagram
  - "largest empty circle centred in region R" → candidates: Voronoi vertices, Voronoi edges ∩ boundary of R, vertices of R
- $O(n^2 \log n)$ construction: each cell = half-plane intersection of $n - 1$ bisectors (nationals-08)
- $O(n^2)$ with Delaunay → dual
- WF 2018 *Panda Preserve*: max over points in a polygon of distance to nearest vertex — exactly the "largest empty circle" pattern

Minkowski sums

$A \oplus B = \{ a + b : a \in A, b \in B \}$. For convex polygons it is convex with $\leq n + m$ vertices.

- Start both at the lowest (then leftmost) vertex
- Edges of $A \oplus B$ = edges of A and B **merged by polar angle**
- Merge like merge-sort using the sign of the cross product of current edges

```
A edges:  →  ↗  ↖  ↙        merged:  →  ↗  ↑  ↖  ←  ↙  ↓
B edges:  ↑  ←  ↓
```

Minkowski sum code

```
void reorder(vector<P>& a) {                        // start at lowest, then leftmost
    int k = 0;
    for (int i = 1; i < (int)a.size(); i++)
        if (a[i].y < a[k].y || (a[i].y == a[k].y && a[i].x < a[k].x)) k = i;
    rotate(a.begin(), a.begin() + k, a.end());
}
vector<P> minkowski(vector<P> a, vector<P> b) {  // strictly convex, CCW
    reorder(a); reorder(b);
    int n = a.size(), m = b.size();
    a.push_back(a[0]); a.push_back(a[1]);
    b.push_back(b[0]); b.push_back(b[1]);
    vector<P> r; int i = 0, j = 0;
    while (i < n || j < m) {
        r.push_back(a[i] + b[j]);
        ll c = (a[i+1] - a[i]).cross(b[j+1] - b[j]);
        if (c >= 0 && i < n) i++;                   // parallel: advance both
        if (c <= 0 && j < m) j++;
    }
    return r;
}
```

Using Minkowski sums

- **Collision:** $A \cap B \neq \varnothing \Leftrightarrow 0 \in A \oplus (-B)$
- **Distance** between convex polygons = distance from 0 to $A \oplus (-B)$
- **Motion planning** of a translating convex robot: grow obstacles by −robot
- **Point in $A \oplus B \oplus C$**: CF 87E *Mogohu-Rea Idol* (sum of three, then O(log n) point-in-convex)
- **Range Minkowski sums**: CF 1195F — #vertices of a sum = #distinct edge directions (normalise by gcd, offline distinct counting)
- Non-convex: decompose into convex pieces, or use a convolution of slopes (expensive)

Other WF-flavoured 3D tasks

- Points on a sphere / great-circle distances: work with unit vectors, angles via `atan2(|u×v|, u·v)` (stable), not `acos`
- 3D segment–segment distance: closest points by clamped parametric minimisation; test parallel case separately
- Polyhedron volume from faces: Σ vol(O, f)/6 with consistent orientation
- Reduce to 2D: project onto a face plane, keep integer coordinates by dropping one axis

Testing geometry like a WF team

- Write the **checker** before the solution: hull ⇒ closed surface + all points inside + Euler
- Generators: tiny coordinate ranges (−2..2) — they create every degeneracy fast
- Also: all coplanar, all collinear, duplicates, n = 4, a regular polygon (cocircular)
- Compare against an $O(n^4)$ brute force for small n
- Time the worst case: points on a sphere / on a paraboloid

Common pitfalls

- `acos(x)` with x = 1 + 1e-16 → NaN; clamp or use `atan2`
- Mixing exact predicates with `double` constructions in the same comparison
- Non-strict visibility (`>= 0`) in the 3D hull ⇒ degenerate faces / broken horizon
- Forgetting that cocircular inputs make Delaunay non-unique — answers must not depend on the choice
- Minkowski with collinear input vertices ⇒ duplicate points; clean input polygons first
- `x*x + y*y` overflow in the lift: check the degree-4 budget

Practice problems

- Luogu **P4724** — 3D convex hull template (surface area)
- SPOJ **CH3D**, HDU **3662** — 3D hull area / face count
- CF **1142C** *U2* — lifting / upper hull of transformed points
- CF **87E** *Mogohu-Rea Idol* — Minkowski sum of three polygons
- CF **1195F** *Geometers Anonymous Club* — Minkowski + distinct directions
- ICPC WF 2018 *Panda Preserve* (Kattis `pandapreserve`) — Voronoi / largest empty circle
- ICPC WF 2015 *Asteroids* (Kattis `asteroids`) — convex polygon intersection over time
- ICPC WF 2017 *Airport Construction* (Kattis `airport`) — robust segment-in-polygon

Summary

- Exact predicates + approximate constructions; count bits for every predicate
- 3D hull: tetrahedron, strict visibility, horizon via edge stamps, $O(n^2)$
- Delaunay = lower hull of lifted points; Voronoi = its dual
- Minkowski sum = merge of edge sequences; turns collision/distance into point queries
- Test with tiny coordinate ranges and structural checkers

Next: Lecture 7 — Burnside/Pólya, Möbius inversion, generating functions, sublinear prime sums